

There will be **NO lectures** on April 3 (Good Friday) and April 10 (non-teaching week).

Workshops next week (March 30 - April 2) will be delivered as usual. **NO** workshops in non-teaching week.

Assignment 1 (worth 15%) will be released next week and due at 23:59 April 16 (Thursday).

An **anonymous survey** of this subject can be found at this link:
<https://go.unimelb.edu.au/smh2>

COMP90038 Algorithms and Complexity

Graphs and Graph Concepts

Junhao Gan

Week 4 Lecture 7

Semester 1, 2026

Graphs and Trees

In this lecture, we define and introduce graphs, trees and their related concepts.

Various problems in real-world applications can be modelled as problems on graphs, and solved by graph algorithms.

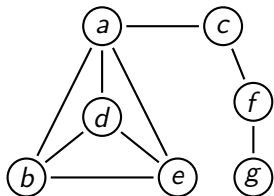
- Road networks, e.g., Google Map
- Social network, e.g., Facebook, LinkedIn
- User-item networks, e.g., Netflix, Amazon, eBay
- Internet webpages, e.g., PageRank
- Many combinatorial optimization problems, e.g., TSP
- GraphRAG in AI

Graphs, Mathematically

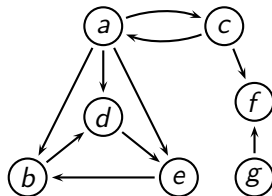
$$G = \langle V, E \rangle$$

V : Set of *nodes* or *vertices*

E : Set of *edges*, where $E \subseteq V \times V$



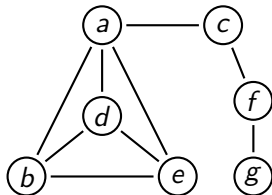
$$\begin{aligned} V &= \{a, b, c, d, e, f, g\} \\ E &= \text{symmetric closure of} \\ &\quad \{(a, b), (a, c), (a, d), \\ &\quad (a, e), (b, d), (b, e), \\ &\quad (c, f), (d, e), (f, g)\} \end{aligned}$$



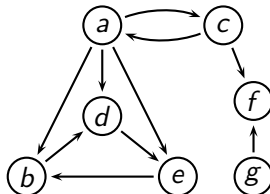
$$\begin{aligned} V &= \{a, b, c, d, e, f, g\} \\ E &= \{(a, b), (a, c), (a, d), \\ &\quad (a, e), (b, d), (c, a), \\ &\quad (c, f), (d, e), (e, b), \\ &\quad (g, f)\} \end{aligned}$$

Graph Concepts

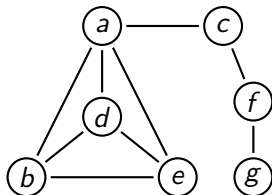
Undirected:



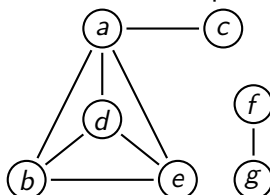
Directed:



Connected:

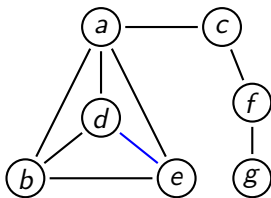


Not connected, two components:



Undirected Graph: Degrees of Nodes

Consider an undirected graph $G = \langle V, E \rangle$.



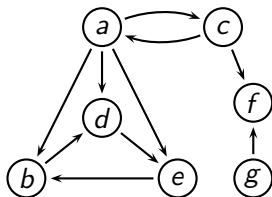
If $(v, u) \in E$, then v and u are **adjacent**, or **neighbours**.

The edge (v, u) is **incident** on, or **connects**, v and u .

The **degree** of node v , denoted by $\deg(v)$, is the *number of edges incident on v* .

Question: What is the relationship between $\sum_{v \in V} \deg(v)$ and the number of edges $|E|$?

Directed Graph: Degrees of Nodes



If there is an edge $(v, u) \in E$, we say there is *an edge from v to u* .

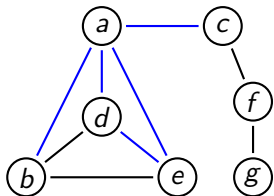
And u is an *out-neighbour* of v , and v is an *in-neighbour* of u .

For a node $v \in V$,

- *in-degree* $\deg_{\text{in}}(v)$ is the number of in-neighbours of v
- *out-degree* $\deg_{\text{out}}(v)$ is the number of out-neighbours of v .

Question: What is the relationship between $\sum_{v \in V} \deg_{\text{in}}(v)$ and the number of edges $|E|$? How about $\sum_{v \in V} \deg_{\text{out}}(v)$?

More Graph Concepts: Paths and Cycles

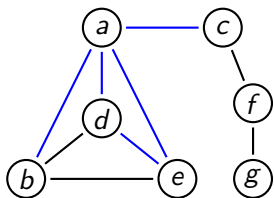


Path $\langle b, a, d, e, a, c \rangle$ shown in blue

A **path** P in $G = \langle V, E \rangle$ is a *sequence of nodes* $P = \langle v_0, v_1, \dots, v_k \rangle$ from V , such that $(v_i, v_{i+1}) \in E$ for all $i = 0, 1, 2, \dots, k - 1$.

We say that these nodes and edges are all **on** the path P .

More Graph Concepts: Paths and Cycles



Path $\langle b, a, d, e, a, c \rangle$ shown in blue

The **length** of a path $P = \langle v_0, v_1, \dots, v_k \rangle$ is defined as the **number of edges** on the path, i.e., k .

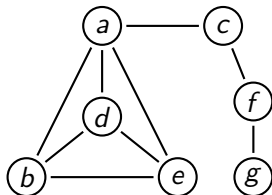
A path is **simple** if there are **no** repeated nodes on it.

A path $P = \langle v_0, v_1, \dots, v_k \rangle$ is a **cycle** if $v_0 = v_k$ with $k > 0$ and no edge is traversed more than once.

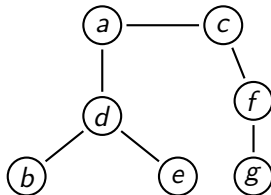
All these definitions of paths and cycles apply to both directed and undirected graphs.

More Graph Concepts

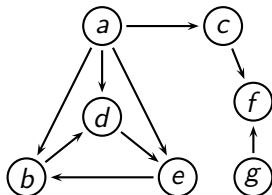
Cyclic:



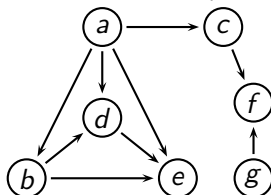
Acyclic:



Directed Cyclic:

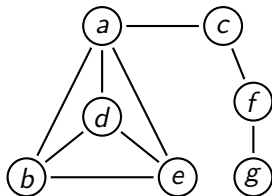


Directed Acyclic Graph (DAG):

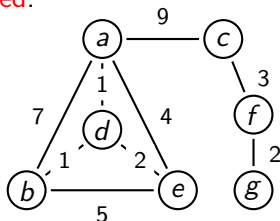


More Graph Concepts

Unweighted:



Weighted:

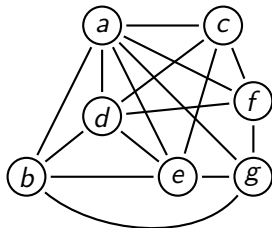


Consider a graph $G = \langle V, E \rangle$; if each edge $(u, v) \in E$ is associated with a *weight*, denoted by $w(u, v)$, then G is a *weighted graph*.

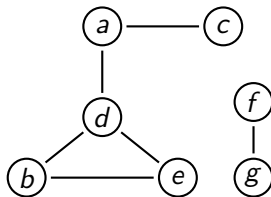
An *unweighted graph* can be considered as a special weighted graph, where each edge has weight equal to 1.

More Graph Concepts

Dense:



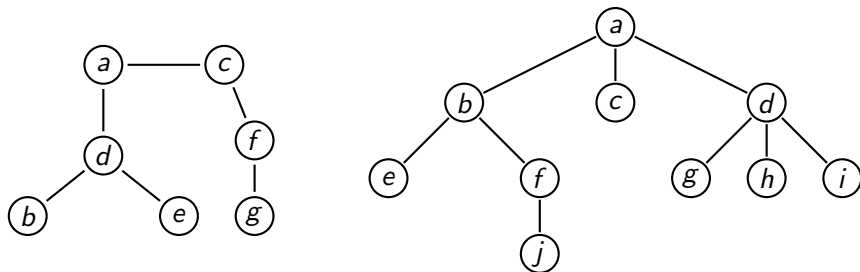
Sparse:



Consider a graph $G = \langle V, E \rangle$; if $|E| \in O(|V|)$, then G is a *sparse graph*. Otherwise, G is a *dense graph*.

Tree

A **tree** is a **connected undirected acyclic graph**.



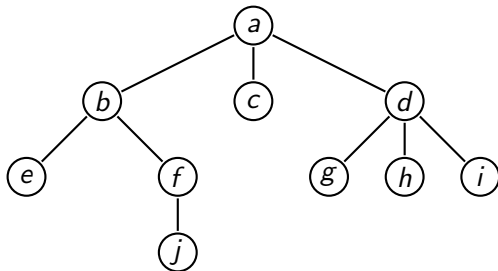
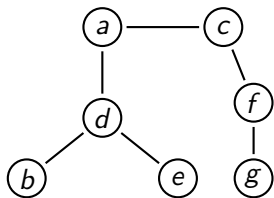
Question: Which one of the above is a tree?

For every pair of nodes u, v in a tree T , there must exist a **unique simple path** between u and v .

Rooted Tree

Consider a tree T and a specified node r in T .

The **rooted tree** of T at r , denoted by T_r , is a tree with nodes organized *level by level* based on the *lengths of their unique simple paths* to the node r . And r is the *root* of T_r .

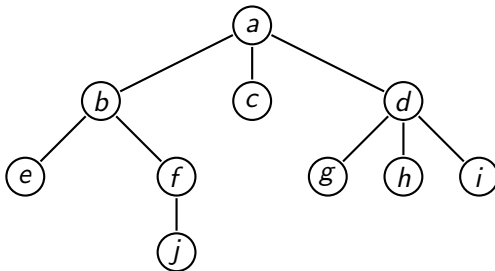


Question: Which of the above is a rooted tree?

Child and Parent

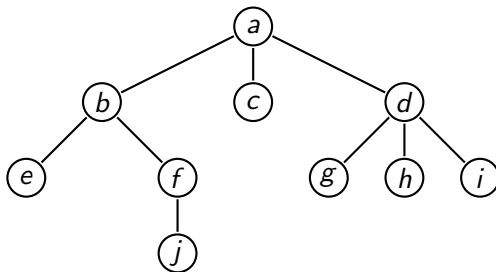
Consider a node v other than the root in a rooted tree; let u be the node adjacent to v that is closer to the root.

We say u is **the parent** of v , and v is **a child** of u .



Leaf

A node v in a rooted tree is a **leaf** if and only if v has no child.



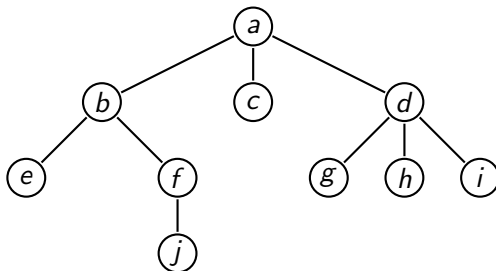
Sub-tree

Consider a node v other than the root r in a rooted tree T .

Removing the edge between v and its parent, the rooted tree T is divided into two (rooted) trees: one is rooted at r and the other rooted at v .

The one rooted at v is defined as the **sub-tree** of T rooted at v .

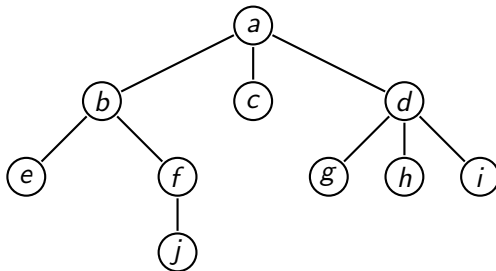
Moreover, we define: the sub-tree of T rooted at the root r is T itself.



Ancestor

For a node v in a rooted tree, all the nodes on the (simple) path from v to the root are **ancestors** of v .

Except v itself, those nodes are v 's **proper ancestors**.

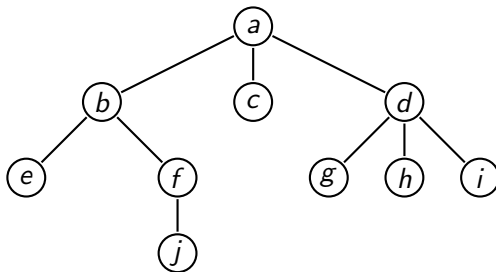


a, b, f, j are all ancestors of j , while only a, b, f are j 's proper ancestors.

Descendant

For a node v in a rooted tree, all the nodes in the **sub-tree rooted at v** are **descendants** of v .

Except v itself, they are v 's **proper descendants**.

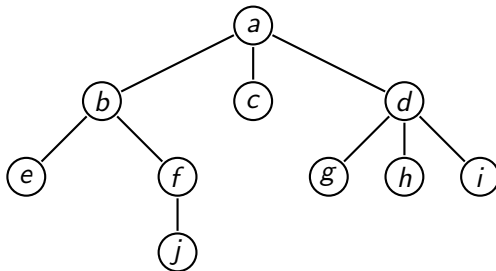


b, e, f, j are all descendants of b , while only e, f, j are b 's proper descendants.

Node Depth

The **depth** of a node v in a rooted tree is the number of *proper ancestors* of v .

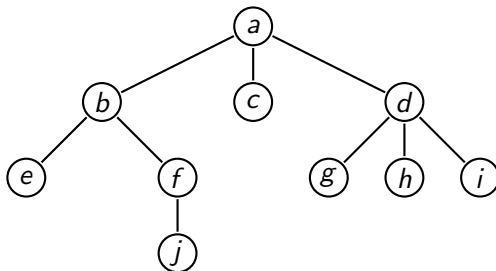
In particular, the depth of the root is 0.



The depth of j is 3 and the depth of g is 2.

Tree Height

The **height of a rooted tree** is defined as the **maximum depth** of the nodes, namely, the **number of edges** on the **longest root-to-leaf path** in the tree.



The maximum node depth is 3, thus, the height of the tree is 3.

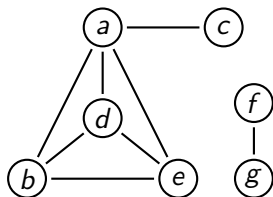
Modelling with Graphs

As we mentioned earlier, graphs find use in all sorts of modelling.

Let us model these applications as graph problems.

- Road networks, e.g., Google Map
- Social network, e.g., Facebook, LinkedIn
- User-item networks, e.g., Netflix, Amazon, eBay
- Internet webpages, e.g., PageRank
- Many combinatorial optimization problems, e.g., TSP
- GraphRAG in AI

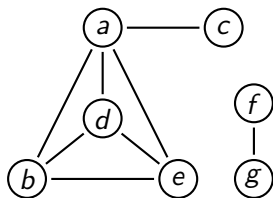
Graph Representations, Undirected Graphs



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>
<i>a</i>	0	1	1	1	1	0	0
<i>b</i>	1	0	0	1	1	0	0
<i>c</i>	1	0	0	0	0	0	0
<i>d</i>	1	1	0	0	1	0	0
<i>e</i>	1	1	0	1	0	0	0
<i>f</i>	0	0	0	0	0	0	1
<i>g</i>	0	0	0	0	0	1	0

The **adjacency matrix** for the graph.

Graph Representations, Undirected Graphs

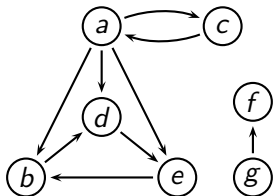


a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

The **adjacency list** representation.

(Assuming lists are kept in sorted order.)

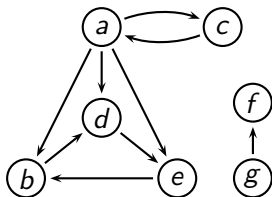
Graph Representations, Directed Graphs



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>
<i>a</i>	0	1	1	1	1	0	0
<i>b</i>	0	0	0	1	0	0	0
<i>c</i>	1	0	0	0	0	0	0
<i>d</i>	0	0	0	0	1	0	0
<i>e</i>	0	1	0	0	0	0	0
<i>f</i>	0	0	0	0	0	0	0
<i>g</i>	0	0	0	0	0	1	0

The **adjacency matrix** for the graph.

Graph Representations, Directed Graphs



<i>a</i>	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
<i>b</i>	$\rightarrow d$
<i>c</i>	$\rightarrow a$
<i>d</i>	$\rightarrow e$
<i>e</i>	$\rightarrow b$
<i>f</i>	
<i>g</i>	$\rightarrow f$

The **adjacency list** representation.

Graph Representations

Each representation has advantages and disadvantages.

Think of some!



Up Next

Graph traversal, in which we get down to the nitty-gritty details of graph algorithms.